

CANNONMAP

Implementation Roadmap

Architecture Version 2.0

Evolutionary refactoring and subsystem delivery plan

Document control	Value
Baseline	CannonMap Software Architecture Specification Version 1.0
Approved additions	Route Family, Commitment, Publication, Confidence Evolution, and Layer 7 Evaluation
Repository baseline	main at 61a9e885; Rider Manager reviewed at d143819
Status	Implementation-ready roadmap; no product code generated
Planning horizon	Incremental, five-year maintainability

Architecture guardrail The architecture is frozen. This roadmap sequences implementation and extraction only. Any deviation requires a documented impossibility, an architecture decision record, and explicit approval.

Executive Summary

CannonMap can reach the approved architecture without a rewrite. The safe path is to establish testable seams around the existing application, extract stable capabilities from app.js in dependency order, introduce offline-first domain stores, and add server-authoritative intelligence processing behind feature flags. Existing Rally Mode, GPS Checkpoints integration, stationary events, Rider Manager, imports/exports, and offline shell remain operational throughout.

- Freeze behavior first: characterize public functions, state mutations, IndexedDB writes, service-worker behavior, and Firebase listener ownership.
- Extract leaf utilities and adapters before orchestration. app.js remains the composition root until late in the program.
- Adopt append-only observations and explicit state machines. Never retrofit inferred state as observed fact.
- Separate Route Families from Route Variants: family aggregate statistics never replace independent variant statistics.
- Represent seven confidence dimensions independently; prohibit a single combined confidence score in contracts and UI.
- Publish intelligence through staged, auditable transitions; Layer 7 evaluates recommendations only after outcomes and never updates models during an active rally.
- Use shadow writes, dual reads, reconciliation reports, and reversible feature flags for every persistent migration.

Reading Guide

Part	Purpose
1–3	Repository assessment, extraction order, and dependency graph
4–6	Persistent schemas, Cloud Functions/APIs, and client runtime architecture
7	Independent delivery milestones with rollback and acceptance gates
8–10	Testing, performance/scaling, and security/privacy controls
11–12	Technical debt, deferrals, and agent-executable implementation order

Part 1 — Repository Assessment

1.1 Current state

Area	Current implementation	Roadmap implication
Application shell	Vanilla JS PWA; index.html, app.css, app.js	Preserve no-build deployment; add ES modules incrementally.
Composition	app.js is ~1,491 lines and owns state, map, persistence, imports, weather, traffic, rally UI, and wiring	Treat app.js as composition root; extract one capability per commit.
Existing modules	gps-checkpoints-feed.js, stationary-events.js, rider-preferences.js	Use their UMD/factory style as transitional seams; migrate to explicit factories.
Persistence	One IndexedDB project store plus localStorage settings/snapshots/preferences	Add versioned domain stores without replacing project storage initially.

Area	Current implementation	Roadmap implication
Realtime	Firebase 8 browser SDK; two RTDB subscriptions owned by feed adapter	Centralize listener ownership; no UI component may subscribe directly.
Offline	Service-worker application shell; last-known local project	Add durable outbox, reconciliation, and opt-in Background Sync.
Tests	Node tests and serial Playwright across phone/desktop profiles	Every extraction must preserve the authoritative serial baseline.
Deployment	Static host with locally vendored browser dependencies	Cloud Functions are additive; PWA remains deployable independently.

1.2 Existing strengths to preserve

- Dependency-free domain modules already exist for stationary-event detection and rider preferences.
- GPS feed normalization separates REST metadata from Firebase event/location subscriptions.
- Runtime dependencies are pinned, vendored, validated, and cached locally.
- Startup readiness is deterministic and optional Firebase/SheetJS failures do not block the base shell.
- The test matrix covers mobile Rally Mode, startup, offline shell, imports/exports, feed normalization, stationary events, and Rider Manager.

1.3 Expected directory structure

```

/
index.html          # stable PWA entry
app.js              # transitional composition root; eventually bootstrap only
src/
  core/              # event-bus, state-store, clock, ids, errors, config
  domain/
    observations/    # collector, quality, observation state machine
    checkpoints/     # lifecycle and checkpoint intelligence
    commitment/      # explainable inference only
    routes/          # families, variants, DNA lifecycle/statistics
    confidence/       # seven independent dimensions + evolution
    intelligence/     # validation, publication, evaluation, notification
    compatibility/    # explainable rider compatibility
    network/         # user-controlled intelligence network
  application/       # use cases/orchestrators; no DOM or Firebase calls
  infrastructure/
    indexeddb/        # schema, migrations, repositories, outbox
    firebase/         # RTDB adapters, auth, rules contract
    sync/             # queue, retry, reconciliation, background sync
    cache/            # aggregate and DNA caches
  ui/
    map/             # Leaflet engine and layer registry
    rally/           # Rally Mode presenters/controllers
    rider-manager/    # DOM rendering only
    planner/         # Geoman and project editing
    integrations/     # GPS Checkpoints, weather, traffic, export
  functions/src/      # Cloud Functions and shared server contracts
  shared/contracts/   # JSON schemas, enums, validation, schema versions
  tests/{unit,integration,browser,offline,performance,fixtures}
  docs/{architecture,decisions,migrations,runbooks}

```

1.4 Coding conventions

- Factories with injected dependencies; no hidden singleton except the top-level composition root.
- Pure domain functions where possible; side effects only in infrastructure and UI adapters.
- Explicit event names in past tense (observation.captured) and commands in imperative form (PublishIntelligence).
- All persisted records carry schemaVersion, createdAt, updatedAt, eventId, and deterministic/idempotency keys.
- All inferences carry evidenceRefs, algorithmVersion, inputs, output, and explanation; observed fields never contain inferred values.
- Use integer milliseconds in storage and ISO-8601 only at external/document boundaries.
- No module imports app.js; app.js imports modules. Lower architecture layers never import higher layers.
- One concern per commit, tests in the same commit, docs/ADR when a durable contract changes.

Part 2 — Refactoring Strategy

Extraction follows the strangler pattern. Each step adds a seam, moves behavior unchanged, runs the full baseline, and leaves a one-commit rollback. New subsystems start only after their prerequisite seams are stable.

Order	Extraction	From app.js	Destination	Proof of equivalence
1	Core utilities	IDs, escaping, geometry math, clocks	src/core + src/domain/geo	Unit vectors; no DOM.
2	Persistence adapter	openDatabase, load/save project, snapshots	src/infrastructure/indexeddb	Round-trip and migration fixtures.
3	State store/event bus	global state mutations and render fan-out	src/core/state-store, event-bus	Mutation log equals existing behavior.
4	Map engine	Leaflet initialization, layer registry, fit/zoom	src/ui/map	Visual snapshots and layer counts.
5	Project/import/export	GPX/project parse, duplicate logic, exports	src/application/project + integrations	Golden fixture byte/semantic comparisons.
6	Competitor/rider presentation	rendering and Rider Manager DOM	src/ui/rally, ui/rider-manager	Existing 35-case Rider Manager matrix.
7	Rally checkpoint workflow	checkpoint state and scoring	src/domain/checkpoints + application/rally	State-machine transition tests.
8	External intelligence	weather, radar, traffic	src/integrations	Mocked contract and abort/retry tests.
9	Bootstrap/wiring	wireUi, renderAll, initialization	src/bootstrap.js; app.js shim	All 85 browser cases unchanged.

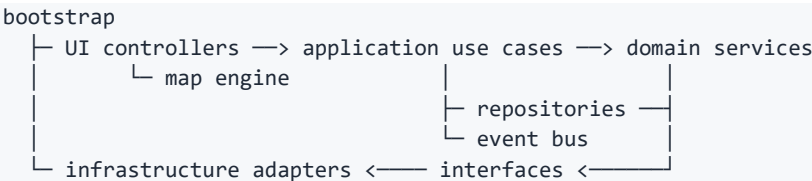
2.1 Extraction rules

- Do not combine extraction with behavior changes, schema migrations, or UI redesign.
- Add contract tests against existing behavior before moving code.
- Keep temporary compatibility facades so index.html and CannonMapTest remain stable.
- Move one dependency direction at a time; reject circular imports.

- Gate new domain subsystems behind disabled-by-default configuration until shadow validation passes.
- Delete legacy paths only after two releases with reconciliation telemetry and rollback rehearsal.

Part 3 — Dependency Graph

3.1 Layer graph



Domain pipeline (upward only):

L0 Raw Observation

- Quality + Commitment inference
- Completed Observation
- Validated Observation
- L1 Crowd Intelligence
- L2 My Intelligence Network
- L3 Compatibility
- L4 Route Family / Variant DNA
- L5 Checkpoint Intelligence
- L6 Co-Driver Publication
- L7 Intelligence Evaluation

3.2 Module dependencies

Module	May depend on	Must not depend on
ObservationCollector	clock, sensor ports, observation repository, event bus	DOM, Firebase SDK, route/intelligence engines
QualityScorer	observation contract, deterministic math	UI, network, publication
CommitmentEngine	validated recent observations, checkpoint geometry, clock	user-entered intent, Route Family output
RouteFamilyEngine	validated completed observations, commitment evidence, confidence dimensions	UI, notifications
ConfidenceEvolutionEngine	evidence ledger, clocks, dimension policies	single aggregate score
PublicationPipeline	validated intelligence, policy, rider context	raw Firebase snapshots
EvaluationEngine L7	published recommendation, later outcome, freeze policy	active-rally model mutation
SyncEngine	repositories, transport, auth, retry policy	DOM and map layers
MapEngine	Leaflet, presentation models, layer registry	Firebase and domain repositories
StateStore	domain snapshots and UI state	direct network or IndexedDB calls

3.3 Critical delivery dependencies

Contracts → Event Bus/Store → IndexedDB v2/Outbox → Observation Capture
Observation Capture → Quality → Commitment → Route Families/Variants
Route Families + Confidence Dimensions → Checkpoint Intelligence

Validated Intelligence → Publication Pipeline → Notifications
 Published Recommendation + Outcome → Layer 7 Evaluation
 Auth + Rules + Idempotency → Production Sync and Cloud Functions
 Performance harness → 500/1000/5000-rider release gates

Part 4 — Database Design

4.1 Firebase RTDB schema

```
events/{eventId}/
  metadata
  activeRally: { status, startsAt, endsAt, modelFreezeId }
  public/
    competitorPresence/{competitorId}
    crowdIntelligence/{scopeKey}/{intelligenceId}
    routeFamilies/{familyId}
    routeVariants/{familyId}/{variantId}
    checkpointIntelligence/{checkpointId}
  private/{uid}/
    observationReceipts/{observationId}
    intelligenceNetwork/{memberId}
    coDriverEvents/{eventId}
    notifications/{notificationId}
    exports/{requestId}

observationIngress/{eventId}/{uid}/{observationId}
observationStatus/{eventId}/{uid}/{observationId}
validatedObservations/{eventId}/{observationId}      # server-only write
evidenceLedger/{eventId}/{evidenceId}                # server-only write
commitmentInferences/{eventId}/{competitorId}/{id}   # server-only write
recommendations/{eventId}/{uid}/{recommendationId}
recommendationOutcomes/{eventId}/{uid}/{recommendationId}
evaluationQueue/{eventId}/{recommendationId}
deadLetters/{functionName}/{yyyyMMdd}/{id}          # admin only
schema/meta/{schemaName}
```

Schema correction Version 1.0 described client-style HTTP verbs over RTDB paths. Version 2.0 treats RTDB as a data service with rules-controlled reads/writes; callable/HTTP Functions provide commands that require authorization, validation, or multi-record transactions.

4.2 Key records

Record	Required identity and relationships	Mutation policy
Observation	observationId; eventId; rider pseudonym; checkpointId?; deviceSessionId	Append-only; correction creates a superseding record.
CommitmentInference	inferenceId; competitorId; checkpointId; evidenceRefs[]; explanation	Server-authored; revised inference links supersededBy.
RouteFamily	familyId; from/to checkpoint; aggregateStats; lifecycle; memberVariantIds[]	Versioned snapshots; merge/split emits lineage.
RouteVariant	variantId; familyId; independentStats; geometryFingerprint; evidenceRefs[]	Independent lifecycle/statistics; never overwritten by family aggregate.
ConfidenceVector	quality, evidenceStrength, inference, historical, current, recency, stability	Dimensions persisted separately with method/version.

Record	Required identity and relationships	Mutation policy
IntelligenceItem	private/completed/validated/crowd/evaluated/notified states; evidenceRefs[]	State-machine transitions only.
RecommendationEvaluation	recommendationId; outcomeRefs[]; prediction; result; error classification	Created after outcome; model update deferred outside active rally.

4.3 Index strategy

- Index observations by eventId+userId+timestamp, eventId+checkpointId+timestamp, syncState+nextAttemptAt, and deviceSessionId+sequence.
- Index variants by eventId+fromCheckpoint+toCheckpoint+lifecycleState and familyId+updatedAt.
- Index intelligence by audienceUid+publicationState+surfaceAfter, and evidence item by source observation.
- Keep RTDB fan-out read paths shallow. Write denormalized public projections only from Cloud Functions.
- Use deterministic IDs for idempotent ingestion; never use client push IDs for retryable commands.

4.4 IndexedDB schema

Store	Key path	Indexes / retention
projects	id	updatedAt; existing store retained
observations	[eventId, observationId]	riderTime, checkpointTime, syncState, sessionSequence; append-only
observationOutbox	idempotencyKey	nextAttemptAt, eventId, state; delete after receipt + retention window
commitmentInferences	[eventId, inferenceId]	competitorCheckpointTime, active
routeFamilies	[eventId, familyId, revision]	checkpointPair, lifecycle, current
routeVariants	[eventId, variantId, revision]	familyId, checkpointPair, lifecycle
confidenceVectors	[eventId, subjectType, subjectId, revision]	updatedAt
intelligenceItems	[eventId, intelligenceId]	stage, audience, surfaceAfter
recommendations	[eventId, recommendationId]	status, createdAt
recommendationEvaluations	[eventId, recommendationId, revision]	evaluatedAt, modelVersion
intelligenceNetwork	[uid, memberId]	updatedAt
syncMeta	key	singleton cursors, schema version, migration checkpoints

4.5 Migration strategy

1. Introduce schema registry and migration runner without changing existing stores.
2. Create new stores in one IndexedDB version upgrade; never transform large records inside onupgradeneeded.
3. Background-copy legacy project competitor/checkpoint data in bounded batches; record a durable migration cursor.
4. Shadow-write new observations while legacy behavior remains authoritative.
5. Run reconciliation counts/hashes and surface non-PII diagnostics.
6. Enable dual-read with new-store preference and legacy fallback.

7. Promote new stores to authoritative behind a remotely disableable flag.
8. Retain legacy fields for two releases; delete only after export, rollback, and offline upgrade tests pass.

Part 5 — Cloud Functions and APIs

5.1 Cloud Function inventory

Function / trigger	Purpose and output	Failure handling
ingestObservation — callable/HTTP	Validate auth/schema/idempotency; write ingress; return receipt/status URL	Reject invalid; duplicate returns original receipt; transient retry.
onObservationIngress — RTDB create	Server re-score quality; write validated observation/evidence ledger	Quarantine invalid; dead-letter after bounded retries.
inferCommitment — observation batch trigger	Infer checkpoint commitment from multiple signals; emit explanation and evidence	Emit no inference when insufficient; never guess.
completeObservation — commitment/transition trigger	Close checkpoint/route observation and mark completed	Idempotent state transition; incomplete remains pending.
updateRouteVariant — validated observation create	Update variant independent statistics and confidence vector	Transactional revision; retry on contention.
reconcileRouteFamily — queue/scheduled	Update family aggregate; propose provisional merge/split with lineage	Keep prior revision on failure; provisional until evidence threshold.
updateCheckpointIntelligence — validated observation create	Maintain checkpoint and sequence aggregates	Rebuildable projection; retry safely.
validateIntelligence — evidence create	Promote completed to validated only when evidence/policy pass	Record rejection reasons; no publication.
publishCrowdIntelligence — validated item create	Anonymize and publish crowd projection	Privacy failure blocks publication.
evaluateForCoDriver — crowd/network change	Personalize/evaluate meaningfulness; create candidate event	No surface when context stale or confidence dimensions insufficient.
dispatchNotification — scheduled/queue	Deliver eligible notification and mark receipt	Deduplicate; exponential retry; expire stale.
recordRecommendationOutcome — callable/trigger	Associate later observed outcome with recommendation	Pending until outcome sufficient; immutable audit record.
evaluateRecommendation — outcome create	Layer 7 accuracy/calibration evaluation	No model mutation during active rally; enqueue post-rally learning.
postRallyModelUpdate — scheduled/admin	Apply approved learning after rally freeze ends	Versioned model, canary, rollback pointer.
recomputeCompatibility — scheduled/on profile	Compute explainable compatibility evidence	Retain previous score if computation fails.
privacyExport / privacyDelete — callable	GDPR/CCPA export or deletion workflow	Authorized, audited, resumable, signed result.
archiveHistoricalDNA — scheduled	Move superseded lineage to historical/archive tier	Mark-and-sweep; never hard-delete active lineage.
deadLetterReplay — admin callable	Validate and replay selected failures	Admin-only; replay idempotency enforced.

5.2 API contracts

Endpoint	Input / output	Auth and idempotency
POST /v1/events/{eventId}/observations	ObservationEnvelope → Receipt	Firebase ID token; Idempotency-Key required.
POST /v1/events/{eventId}/recommendations/{id}/ack	{acknowledgedAt, clientEventId} → Event state	Owner only; repeat returns same state.
GET /v1/events/{eventId}/bootstrap	Versioned checkpoints, current DNA, cursors → ETag response	Event membership; cacheable private response.
GET /v1/events/{eventId}/sync?cursor=	Ordered deltas + nextCursor	Membership; bounded page size.
PUT /v1/users/{uid}/network/{memberId}	Network member → revision	Owner only; If-Match revision.
DELETE /v1/users/{uid}/network/{memberId}	Tombstone → revision	Owner only; idempotent.
POST /v1/privacy/export	Scope → requestId	Recent auth; rate limited.
POST /v1/privacy/delete	Confirmation token → requestId	Recent auth; delayed/reversible window.

- All contracts use JSON Schema, schemaVersion, requestId, traceId, and explicit error codes.
- 429 responses include retryAfterMs; clients apply full-jitter exponential backoff.
- No API exposes raw cross-rider locations or personal contact data.
- REST metadata from GPS Checkpoints remains isolated in the existing adapter and is not treated as CannonMap authority.

Part 6 — Client Architecture

6.1 Map engine

- Own one Leaflet map and one LayerRegistry keyed by layer type and entity ID.
- Consume presentation models only; never subscribe to Firebase or read IndexedDB directly.
- Apply incremental add/update/remove operations instead of clearing all competitor layers at scale.
- Use viewport filtering, requestAnimationFrame batching, and canvas/WebGL-compatible point paths where practical.
- Preserve the existing satellite base-layer choice and stationary-event overlays.

6.2 Event bus

A synchronous in-process bus carries immutable domain events. Handlers enqueue asynchronous work but do not block publishers. Events include eventId, entityId, occurredAt, correlationId, causationId, schemaVersion, and payload. Development builds detect duplicate subscription keys.

6.3 State store

- Normalized slices: session, project, map, rally, competitors, riderPreferences, observations, intelligence, sync, and UI.

- Reducers are pure; commands invoke repositories/services and then dispatch domain events.
- Feed records, persisted UI preferences, Leaflet layer objects, and stationary-event records remain distinct.
- Selectors compute view models; DOM controllers never mutate domain objects.

6.4 Caching, offline queue, and background sync

capture → IndexedDB observations (transaction)
 → observationOutbox (same transaction)
 → optimistic local event
 online/sync → batch by event/session → authenticated ingest
 → receipt persisted → outbox acknowledged
 → server deltas reconciled by cursor
 failure → nextAttemptAt + full jitter → bounded retries → local dead letter
 service worker → application shell + sync wake-up only
 foreground SyncEngine → authoritative queue owner

Browser constraint Background Sync is not uniformly available, especially on iOS. The foreground SyncEngine is mandatory; service-worker sync is an opportunistic accelerator, never the sole delivery mechanism.

Part 7 — Incremental Milestones

M0 Baseline and architecture guardrails

Field	Plan
Goal	Lock tests, contracts, dependency rules, flags, and observability.
Files modified	docs/*; tests/architecture; scripts/check-boundaries.mjs
Complexity	Low
Regression risk	Low
Testing required	Run current Node/Playwright/offline/vendor baselines.
Rollback	Revert documentation/test-only commit.
Acceptance criteria	Baseline recorded; forbidden dependencies fail CI.

M1 Core seams

Field	Plan
Goal	Extract clock, IDs, geometry, errors, event bus, and state store with compatibility facade.
Files modified	src/core/*; app.js; unit tests
Complexity	Medium
Regression risk	Medium
Testing required	Golden utility tests; browser matrix.
Rollback	Restore app.js imports through facade.
Acceptance criteria	No behavior/diff change; no cycles.

M2 IndexedDB v2 foundation

Field	Plan
-------	------

Field	Plan
Goal	Add schema registry, repositories, outbox, migration checkpoints.
Files modified	src/infrastructure/indexeddb/*; tests/integration
Complexity	High
Regression risk	High
Testing required	Upgrade/rollback/crash/offline fixtures.
Rollback	Disable v2 flag; legacy project store remains authoritative.
Acceptance criteria	Interrupted migration resumes; no data loss.

M3 Map engine extraction

Field	Plan
Goal	Move Leaflet ownership and incremental layer registry.
Files modified	src/ui/map/*; app.js
Complexity	Medium
Regression risk	Medium
Testing required	Layer counts, screenshots, 100/500 rider perf.
Rollback	Repoint facade to legacy functions.
Acceptance criteria	Existing UI and overlays remain equivalent.

M4 Project and Rally extraction

Field	Plan
Goal	Move imports/exports, checkpoint workflow, presenters, and DOM controllers.
Files modified	src/application/*; src/domain/checkpoints/*; src/ui/rally/*
Complexity	High
Regression risk	High
Testing required	GPX/project golden tests; complete Rally matrix.
Rollback	One module extraction reverted per commit.
Acceptance criteria	app.js becomes wiring; behavior unchanged.

M5 Observation capture

Field	Plan
Goal	Append-only capture, local quality vector inputs, durable queue.
Files modified	src/domain/observations/*; sync/*
Complexity	High
Regression risk	High
Testing required	Sensor simulations, crash recovery, battery soak.
Rollback	Flag off collector; retain existing GPS display.
Acceptance criteria	Every accepted sample locally durable <50 ms target.

M6 Auth and secure ingestion

Field	Plan
-------	------

Field	Plan
Goal	Firebase Auth, Rules, ingest function, receipts, idempotency.
Files modified	functions/*; firebase rules; adapters
Complexity	High
Regression risk	High
Testing required	Emulator rules, replay, abuse tests.
Rollback	Keep upload disabled; local capture continues.
Acceptance criteria	Users write only own ingress; duplicate safe.

M7 Commitment Engine

Field	Plan
Goal	Explainable multi-signal inference and explicit lifecycle.
Files modified	src/domain/commitment/*; functions/inferCommitment
Complexity	High
Regression risk	High
Testing required	Trace fixtures, false-positive/negative review.
Rollback	Shadow only; no downstream consumption.
Acceptance criteria	Every inference explains evidence; never marked observed.

M8 Route Family Engine

Field	Plan
Goal	Families, variants, independent/aggregate stats, provisional merge/split.
Files modified	src/domain/routes/*; functions/routes/*
Complexity	Very High
Regression risk	High
Testing required	Deterministic replay, lineage, contention, scale.
Rollback	Shadow projections; retain prior revision pointer.
Acceptance criteria	No statistic loss; merge/split reversible and explainable.

M9 Confidence Evolution

Field	Plan
Goal	Seven independent dimensions and time evolution policies.
Files modified	src/domain/confidence/*; shared/contracts
Complexity	High
Regression risk	High
Testing required	Dimension-specific monotonicity/decay tests.
Rollback	Read old confidence as migration input only.
Acceptance criteria	No single score; each dimension has provenance.

M10 Checkpoint and compatibility

Field	Plan
-------	------

Field	Plan
Goal	Checkpoint/sequence aggregates; explainable network compatibility.
Files modified	domain/checkpoints; compatibility; network
Complexity	High
Regression risk	High
Testing required	Aggregate rebuild and suggestion-only tests.
Rollback	Disable suggestions/projections.
Acceptance criteria	Network never auto-mutated.

M11 Publication pipeline

Field	Plan
Goal	Private→completed→validated→crowd→Co-Driver→notification.
Files modified	domain/intelligence; functions/publication; UI
Complexity	Very High
Regression risk	High
Testing required	Privacy gates, expiry, dedupe, offline delivery.
Rollback	Publication flag off; retain private items.
Acceptance criteria	Only validated, meaningful, timely items surface.

M12 Layer 7 evaluation

Field	Plan
Goal	Post-outcome recommendation evaluation and post-rally learning queue.
Files modified	domain/evaluation; functions/evaluation
Complexity	Very High
Regression risk	High
Testing required	Outcome linkage, freeze enforcement, rollback model.
Rollback	Evaluation read-only; model update flag off.
Acceptance criteria	No active-rally model mutation; audit complete.

M13 Scale and production hardening

Field	Plan
Goal	5000-rider projections, partitioning, operations, recovery.
Files modified	performance; runbooks; monitoring
Complexity	Very High
Regression risk	High
Testing required	Load, chaos, battery, recovery drills.
Rollback	Capacity flag/circuit breakers; prior model version.
Acceptance criteria	SLOs met at approved tier; runbooks rehearsed.

Part 8 — Testing Strategy

Test layer	Coverage	Release gate
------------	----------	--------------

Test layer	Coverage	Release gate
Unit	Pure geometry, quality, state machines, confidence dimensions, family/variant statistics, explanations	Deterministic; mutation and boundary coverage.
Contract	JSON schemas, event envelopes, RTDB projections, APIs, migration versions	Producer/consumer compatibility in CI.
Integration	IndexedDB transactions/upgrades, outbox, Firebase emulator, Function idempotency	Crash/retry/replay fixtures pass.
Browser regression	Planner, Rally, Rider Manager, imports/exports, stationary events, startup	Authoritative serial matrix remains green.
Offline	Cold cached start, foreground reconnect, optional Background Sync, partial sync, clock skew	No data loss; UI labels staleness.
Sync/chaos	Duplicate, reorder, timeout, auth expiry, quota, partial Function failure	Converges to one idempotent result.
Performance	100/500/1000/5000 riders; 1k/10k/100k observations; map pan/zoom	Tier-specific budgets below.
Security/privacy	Rules emulator, schema fuzzing, rate limits, cross-user reads, export/delete	Zero unauthorized reads/writes.
Evaluation	Recommendation/outcome linkage, calibration, active-rally freeze	No learning mutation while active.

8.1 Required test fixtures

- Recorded but anonymized observation sequences: stationary, highway, urban traffic, GPS dropout, checkpoint miss, U-turn, and route divergence.
- Family evolution cases: new variant, borderline provisional, merge, split, supersede, historical replay.
- Confidence vectors with independent changes: fresh/weak, old/stable, strong/low-quality, high-quality/sparse.
- Publication cases covering every rejection reason and notification dedupe/expiry.
- Model-freeze cases spanning rally start/end boundaries and delayed outcomes.

Part 9 — Performance and Scalability

Scale tier	Client strategy	Server/data strategy	Gate
100 riders	All current markers; incremental updates; 5–15 min local trail window	Single event shard; direct public projection	60 fps median; <300 ms UI event.
500 riders	Viewport culling; canvas markers; coalesced 1 Hz UI updates	Partition by event and projection type	<150 MB working set; no long task >50 ms p95.
1,000 riders	LOD summaries; selected trails only; worker statistics	Event+region buckets; queued aggregation	Initial usable map <3 s on target phone.
5,000 riders	Do not render all entities; tile/grid summaries and subscription windows	Mandatory sharding, fan-out projections, backpressure	Load test only until field need approved; graceful degradation.

9.1 Resource budgets

- Memory: cap per-rider in-memory breadcrumbs; page historical data from IndexedDB; release Leaflet layers and listeners deterministically.
- CPU: batch observation calculations; move replay/statistics to Web Worker after profiling proves need; avoid JSON deep clones on hot paths.

- Network: delta cursors, ETags, compact projections, subscription scope, and exponential backoff; never sync unchanged family revisions.
- Battery: adaptive sampling by movement/device state; pause nonessential evaluation in background; no high-frequency polling.
- Storage: retention policies by store; preserve auditable server evidence while compacting local derived caches.
- Measure on representative iPhone hardware, not desktop emulation alone.

Part 10 — Security and Privacy

10.1 Firebase Rules principles

- Default deny. Validate event membership, uid ownership, schemaVersion, allowed keys, numeric bounds, timestamp skew, and record immutability.
- Clients write only observationIngress under their uid, acknowledgements, and their Intelligence Network.
- Only Admin SDK writes validated observations, evidence, Route DNA, crowd intelligence, evaluations, and public projections.
- Public readers receive anonymized aggregates with k-anonymity/minimum-evidence thresholds; never raw rider histories.
- Rules and Functions share generated schema constants but are tested independently in the Emulator Suite.

10.2 Authentication and abuse prevention

- Support anonymous auth for low-friction local use; require account linking for cross-device recovery, export, or durable network membership.
- Use App Check, per-uid/event quotas, request-size limits, monotonic device sequence checks, and idempotency keys.
- Reject impossible coordinates/speeds and quarantine suspicious streams; do not silently delete evidence.
- Rate-limit notification acknowledgements, privacy operations, and administrative replay.

10.3 Privacy

- Use event-scoped pseudonymous rider IDs in analytical stores; keep identity mapping in a restricted boundary.
- Opt-in location sharing is explicit, revocable, and limited to active events.
- Strip contact information, tokens, device identifiers, and raw location from public intelligence.
- Provide export/delete workflows with audit records, retention policy, and derived-data recomputation.
- Threat-model inference leakage: small cohorts, unique routes, timing correlation, and malicious network membership.

Part 11 — Technical Debt and Deferrals

11.1 Current risks

Risk	Impact	Treatment
Monolithic app.js and global mutable state	Regression coupling and difficult tests	Milestones M1–M4; do not add new domains to app.js.
Single project-oriented IndexedDB store	No durable append-only/outbox semantics	M2 additive schema; legacy retained.
Firebase v8 namespaced SDK	Long-term maintenance/security upgrade pressure	Adapter boundary first; SDK migration later, separately.
Static service-worker cache list	Manual drift risk	Generate manifest from pinned assets/modules.
Global competitor layer rebuild	CPU/memory pressure at scale	Incremental LayerRegistry in M3.
Ambiguous historical confidence formula	Conflicts with seven dimensions	Deprecate aggregate field; dimension migration in M9.
No server code/rules in repository	Architecture cannot be enforced	M6 establishes functions/rules/emulator.

11.2 Future risks

- Premature route-family merge/split could destroy lineage; all classifications begin provisional and remain reversible.
- Recommendation feedback can create self-reinforcing bias; evaluate outcomes separately and update only between rallies.
- Large-event Firebase fan-out and hot keys require production load proof before 1,000+ rider claims.
- Offline clients may return with old schemas/models; contracts need multi-version reads and bounded support windows.
- Explainability payloads can expose private evidence; separate private evidence from public explanation summaries.

11.3 Do not implement yet

- No framework rewrite, TypeScript conversion, bundler migration, or UI redesign during domain extraction.
- No machine-learning model until deterministic statistical baselines, evidence ledgers, and Layer 7 evaluation are stable.
- No active-rally online learning or automatic Intelligence Network edits.
- No fuel inference, stop-cause classification, route recommendations, or crowd hazard reporting.
- No 5,000-rider client rendering; build server projections and graceful degradation first.
- No Firebase SDK upgrade bundled with schema migration or auth rollout.
- No deletion of legacy stores/fields before two-release rollback window closes.

Part 12 — Numbered Agent-Executable Implementation Order

9. Record current main SHA, test commands, test totals, runtime assets, and rollback tag in docs/BASELINE.md.

10. Add architecture dependency rules and a CI script that rejects imports from lower layers to higher layers.
11. Add shared schemaVersion constants, domain enums, timestamp policy, error codes, and contract validation tests.
12. Extract clock and ID providers behind compatibility functions; preserve all callers.
13. Extract geo math and normalization utilities; add golden tests from current behavior.
14. Introduce EventBus with duplicate-subscription diagnostics; do not connect production flows yet.
15. Introduce StateStore with immutable slices and selectors; mirror current state in shadow mode.
16. Add IndexedDB schema registry and migration journal without changing the existing project store.
17. Create new v2 stores and indexes; add interrupted-upgrade and quota failure tests.
18. Implement repository interfaces and IndexedDB adapters for observations, outbox, DNA, intelligence, and sync metadata.
19. Add durable outbox transaction helper and foreground retry scheduler with injected clock/randomness.
20. Extract Leaflet MapEngine and LayerRegistry through an app.js facade.
21. Convert competitor layers to incremental keyed updates; retain existing rendering semantics.
22. Extract project persistence and snapshot behavior; compare serialized fixtures.
23. Extract GPX/project import normalization and export services; retain golden outputs.
24. Extract Rally checkpoint state machine and scoring without UI changes.
25. Extract Rally presenters/controllers and Rider Manager rendering; preserve listener ownership.
26. Extract weather/radar/traffic adapters with abort, timeout, and response-schema tests.
27. Reduce app.js to bootstrap/composition and compatibility exports; run full regression matrix.
28. Define ObservationEnvelope and append-only Observation domain model.
29. Implement ObservationCollector writing observation and outbox atomically.
30. Implement deterministic local QualityScorer inputs and separate observationQuality dimension.
31. Add Firebase Auth/App Check bootstrap behind disabled feature flag.
32. Create Firebase Rules and Emulator Suite tests with default-deny ownership validation.
33. Implement idempotent ingestObservation Function and client transport adapter.
34. Implement receipt/delta reconciliation and offline reconnect tests.
35. Implement Commitment state machine and evidence signal contracts.
36. Build CommitmentEngine in shadow mode with human-readable explanations and replay fixtures.
37. Define RouteFamily, RouteVariant, revision, lineage, and provisional classification contracts.
38. Implement independent RouteVariant statistics with deterministic replay.
39. Implement RouteFamily aggregate statistics without overwriting variant statistics.
40. Implement provisional merge/split proposals and reversible lineage transitions.
41. Add server Functions for validated observations, variant updates, and family reconciliation.
42. Define seven-dimension ConfidenceVector and prohibit aggregate confidence in schema validation.
43. Implement recency/stability evolution policies and dimension-specific provenance.
44. Implement Checkpoint/Sequence intelligence projections as rebuildable Functions.
45. Implement user-controlled Intelligence Network repositories and secure CRUD.
46. Implement explainable CompatibilityEngine suggestions; verify no automatic network mutation.
47. Define IntelligenceItem state machine from private observation through notification.
48. Implement validation/anonymization gates and crowd publication projections.
49. Implement Co-Driver meaningfulness evaluation and notification dedupe/expiry.
50. Implement acknowledgement sync and offline-safe event state.
51. Define RecommendationOutcome and RecommendationEvaluation contracts.

52. Implement Layer 7 evaluation after outcomes with active-rally model freeze enforcement.
53. Implement post-rally versioned model update queue and rollback pointer; keep disabled.
54. Add 100/500/1000-rider performance fixtures and device memory/battery profiling.
55. Add 5,000-rider server projection/load test and client graceful-degradation proof.
56. Add operational dashboards, dead-letter runbooks, privacy export/delete, and disaster recovery rehearsal.
57. Run a two-release shadow/dual-read period; reconcile, rehearse rollback, then remove approved legacy paths in separate commits.

Release Governance

Gate	Required evidence
Extraction gate	No behavior change; full existing serial matrix green; one-commit rollback.
Persistence gate	Upgrade, interruption, quota, old-client, export, and rollback fixtures pass.
Server gate	Emulator Rules/Functions tests; idempotency and dead-letter recovery demonstrated.
Intelligence gate	Evidence/explanation audit; privacy gate; no speculation represented as fact.
Active rally gate	Model/config freeze ID recorded; rollback and offline operation tested.
Scale gate	Representative device and server load results meet published tier budgets.
Legacy removal gate	Two releases reconciled; telemetry clean; user data export and rollback snapshot retained.

Architecture Decision Summary

- The Version 1.0 seven-layer pipeline is extended with Layer 7 Intelligence Evaluation; dependencies remain upward-only.
- Route DNA is owned by Route Families; Route Variants retain independent statistics and lineage.
- Commitment is inferred from evidence, never directly observed, and every inference is explainable.
- Intelligence follows private → completed → validated → crowd → Co-Driver evaluation → notification.
- Observation Quality, Evidence Strength, Inference Confidence, Historical Confidence, Current Confidence, Recency, and Stability remain independent.
- Recommendation learning occurs only after outcomes and never mutates models during an active rally.
- The PWA remains Vanilla JavaScript and offline-capable; evolution is incremental and reversible.

Definition of done The roadmap is complete when each subsystem is independently testable, every persistent transition is versioned and replayable, every inference is evidence-backed, offline capture cannot lose accepted observations, and the current production behaviors remain protected by the serial regression matrix.